SOFTWARE REQUIREMENTS SPECIFICATION
YallaFix: Smart Campus Facility Management System

German International University (GIU)
INCS 617 - Software Engineering for Business
Informatics Summer Semester 2026

Team: YallaFix

Nadeen El Khalifa (13004534) | Mariam Ashraf (13006964) | Malak Gharib (13007525) |
Nour Samir (13006170) | Ganna Hamza (13003623) | Abdelrahman El Khatib (13002667)

Tutorial: Tutorial 4 - Hassan Usama

Document Version: 1.2
Date: May 2026

Table of Contents

1. Introduction

YallaFix is a mobile-based facility complaints management system designed to enable members of the GIU community to report infrastructure, maintenance, sustainability, cleanliness, and safety-related issues on campus. The system provides an easy and structured method for submitting complaints by uploading a photo, adding a description, and specifying the issue location.

Facility Management personnel can view complaints, assign workers, and track their status until resolution. The application enhances communication between community members and the Facility Management team, ensuring faster issue resolution and improved campus conditions.

1.1 Purpose

This document defines the complete functional and non-functional requirements for YallaFix, a mobile application for streamlined facility management at GIU.

1.2 Intended Audience

- Development Team
- Project Managers
- Quality Assurance Engineers
- Stakeholders and End Users

2. Product Vision & Scope

2.1 Vision

Create an interactive, transparent, and efficient platform for handling facility-related complaints at GIU. The system aims to reduce response times, improve coordination between users and Facility Management, and promote community engagement through social features such as confirmations, comments, and notifications.

2.2 Scope

The scope of the system includes reporting and managing physical campus issues only, such as maintenance, cleanliness, sustainability, and safety hazards. The system does NOT handle academic, administrative, or personal complaints.

2.3 Key Features

- Multi-role complaint submission and management
- Photo-based issue documentation
- Real-time notifications
- Issue tracking and status updates
- Worker assignment management
- Community confirmation/upvoting
- Duplicate issue detection and merging

3. Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
JWT	JSON Web Token - authentication mechanism
Ticket/Complaint	Digital record representing a facility-related issue
Reporter	GIU community member submitting a complaint
Manager	Facility Management administrator handling complaints
Worker	Technician assigned to resolve complaints
SLA	Service Level Agreement
ERD	Entity-Relationship Diagram
RESTful API	Architectural style for building web services

4. Functional Requirements

4.1 User Roles

The system supports three distinct user roles, each with specific permissions and responsibilities:

4.2 Community Member (Reporter)

- **FR-01:** The system shall allow users to create a new complaint.
- **FR-02:** The system shall allow users to upload an image with the complaint.
- **FR-03:** The system shall allow users to select the issue location.
- **FR-04:** The system shall allow users to categorize the complaint.
- **FR-05:** The system shall allow users to confirm existing complaints.

- **FR-06:** The system shall allow users to comment on complaints.
- **FR-07:** The system shall notify users when complaint status changes.

4.3 Manager (Facility Admin)

- **FR-08:** The system shall allow managers to view all complaints.
- **FR-09:** The system shall allow managers to assign workers to complaints.
- **FR-10:** The system shall allow managers to update complaint status.
- **FR-11:** The system shall allow managers to merge duplicate complaints.
- **FR-12:** The system shall allow managers to post official updates.

4.4 Worker

- **FR-13:** The system shall notify workers when assigned a complaint.
- **FR-14:** The system shall allow workers to view assigned tasks.
- **FR-15:** The system shall allow workers to upload proof of completion.
- **FR-16:** The system shall allow workers to update task progress.

5. Use Cases & UML Diagram

5.1 Use Case Overview

The following diagram illustrates the complete UML Use-Case model for YallaFix, showing all actors and their interactions with the system.

5.2 YallaFix UML Use-Case Diagram

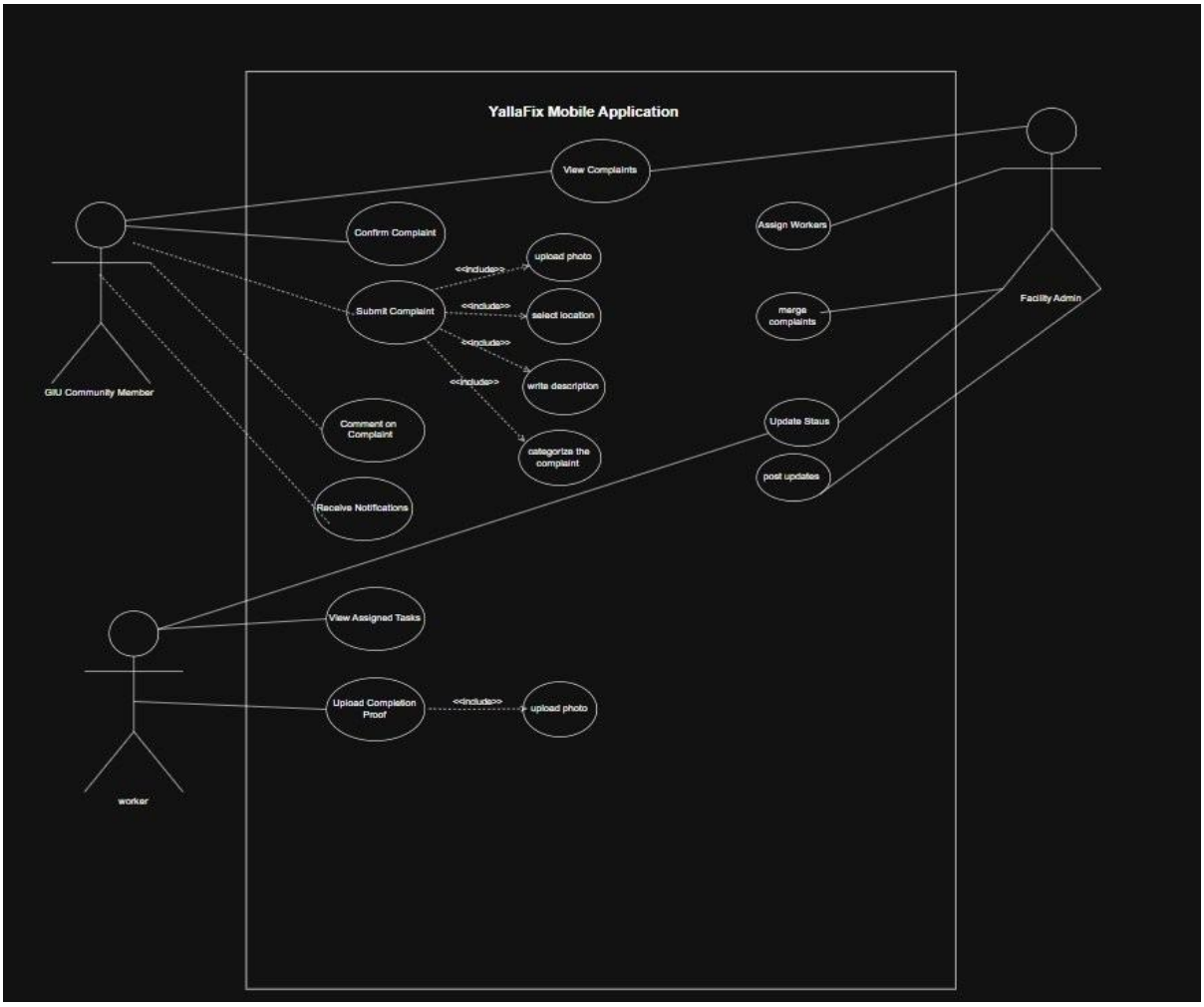

Figure 5.1: YallaFix UML Use-Case Diagram

5.3 Actors

GIU Community Member: Reports facility issues, confirms existing complaints, views status updates

Facility Admin (Manager): Manages all complaints, assigns workers, updates statuses, merges duplicates

Worker: Views assigned tasks, updates progress, uploads completion proof

5.4 Main Use Cases

- Submit Complaint: Upload photo, select location/category, add description
- View Complaints: Display all or filtered complaints
- Confirm Issue: Community upvote/support for complaint
- Assign Worker: Manager assigns technician to complaint
- Update Status: Change complaint status through lifecycle
- View Assigned Tasks: Worker sees their assigned complaints
- Upload Proof: Worker uploads completion photo
- Merge Duplicates: Manager combines duplicate issues
- Receive Notifications: System alerts on status/assignment changes

6. Non-Functional Requirements

Requirement	Details
Response Time	Page load times under 3 seconds
Concurrent Users	Support for at least 500 simultaneous users
API Security	Rate limiting and input validation on all endpoints
Backup & Recovery	Automated daily backups with disaster recovery plan
Usability	Intuitive UI allowing complaint submission in under 1 minute
Reliability	Robust handling of image uploads under unstable network

7. Technical Constraints

- Mobile application must be developed using React Native with Expo
- Backend must be developed using Node.js with RESTful APIs
- Database must use PostgreSQL
- Cloud-based image storage must be implemented (Supabase Storage)
- System must strictly handle facility-related complaints only
- JWT-based authentication for security

8. Database Schema (Entity-Relationship Diagram)

The database consists of the following tables: Users, Complaints, Comments, Notifications, Categories, Locations, Assignments, and Activity Logs.

8.1 Core Entities

- Users: Stores user accounts with role-based access control
- Complaints: Core table for facility issues
- Comments: User comments and official updates
- Notifications: Real-time alerts for users
- Assignments: Tracks worker assignments to complaints
- Categories: Predefined issue categories
- Locations: Campus locations (buildings, floors, rooms)
- Activity Logs: Audit trail for all system actions

9. Technology Stack

Layer	Technology
Mobile Frontend	React Native (Expo SDK 54)
Backend Server	Node.js with Express.js
Database	PostgreSQL (hosted on Supabase)

Authentication	JWT (JSON Web Tokens) + Bcrypt
Image Storage	Supabase Storage (Cloud Object Storage)
Version Control	Git & GitHub

10. Future Scope of the Project

- Heatmap visualization of frequent issue locations
- SLA-based automatic prioritization system
- Analytics dashboard for performance tracking
- Gamification features for active contributors
- AI-based duplicate detection system

11. Document History

Version	Date	Author	Changes
1.0	May 16, 2026	Team YallaFix	Initial release
1.2	May 16, 2026	Team YallaFix	Added actual UML diagram image, corrected Abdelrahman's ID